gemm_local_sram

Scratchpad chia bank, ping-pong

Port list, bố cục logic, tổ chức vật lý, ánh xạ địa chỉ, timing

Bộ tăng tốc GEMM INT8 trên Chipyard SoC

Module	`gemm_local_sram` (instance `u_sram` trong `gemm_accel_top`)
Module con	`gemm_spm_ram` (RAM một cổng chung), `gemm_spm_a`, `gemm_spm_b`, `gemm_spm_bias`, `gemm_spm_c`
Cấu hình	Bản đầy đủ: NB = 2 bank cho mỗi buffer (`HAS_PINGPONG` = 1). NB = 1 cho bản cơ bản
Tuân theo	Spec hệ thống v1.0: mục 4.3 (OWN-01, OWN-02), mục 5.4 (SPM-01 … SPM-06), mục 7.3 (PP-07). Spec `gemm_systolic_array` mục 3 (cổng A, B), spec `gemm_vpu` mục 3 (cổng BIAS, C, bảng 3-3)
Kèm theo	`gemm_sram_model.py`: mô hình ánh xạ; kiểm song ánh, không xung đột sub-bank, nạp/đọc/ghi khứ hồi với P = 4, 8 và NB = 1, 2

Mục lục

1 Chức năng **3**

2 Tham số **3**

3 Port list **3**

4 Bố cục logic **4**

5 Tổ chức vật lý và ánh xạ **5**

5.1 Truy cập phía tính 5

5.2 Truy cập phía DMA 5

5.3 Chọn bên truy cập 6

6 Module con **6**

7 Timing **7**

8 Reset, hiện thực, kiểm chứng **7**

1 Chức năng

Scratchpad giữ bốn buffer logic A, B, C, BIAS, mỗi buffer có NB bank. Phía tính (array, VPU) cần đọc hoặc ghi P phần tử trong một chu kỳ, nên mỗi bank được chia thành nhiều **sub-bank** một cổng sao cho mỗi truy cập song song chạm các sub-bank khác nhau. Phía DMA đọc hoặc ghi một word mỗi chu kỳ. Hai phía dùng hai bank khác nhau (ping-pong) hoặc hai pha khác nhau (không ping-pong), nên không bao giờ tranh nhau một sub-bank.

Bảng 1.1: Chức năng

ID	Yêu cầu
SRM-F01	Mỗi bank có dung lượng đúng SPM-01: A 256 word, B 256 word, C 1024 word, BIAS 32 word; tổng NB $\times$ 1568 word (NB = 2: 12,25 KiB)
SRM-F02	Mọi cổng đọc có độ trễ cố định 1 chu kỳ; mọi lệnh ghi có hiệu lực ở cạnh clock cuối chu kỳ có lệnh; không có backpressure (SPM-04, SPM-05)
SRM-F03	Mỗi sub-bank vật lý là RAM một cổng đồng bộ. Bank b của buffer X chỉ bị truy cập bởi một phía trong một chu kỳ (PP-07); hai bank khác nhau được truy cập độc lập cùng chu kỳ
SRM-F04	Nội dung không được khởi tạo bởi `rst_n` hay `soft_clr` (SPM-06)

2 Tham số

Bảng 2.1: Tham số

Tham số	Mặc định	Ý nghĩa
`P`	8	4 hoặc 8
`HAS_PINGPONG`	1	1: NB = 2; 0: NB = 1
`NB, Q, LP`	2, 2, 3	localparam: số bank; Q = P/4 (số sub-bank của B); LP = $\log_2 P$

3 Port list

Bảng 3.1: Port của `gemm_local_sram`

Port	Rộng	Chiều	Nối với	Mô tả
`clk, rst_n, i_soft_clr`	1 $\times$ 3	In	top, FSM	
Cổng ghi phía DMA (LOAD_A, LOAD_B, LOAD_BIAS)				
`i_dw_valid`	1	In	DMA	Ghi một word
`i_dw_buf`	2	In	DMA	0: A, 1: B, 2: BIAS
`i_dw_bank`	1	In	DMA	Bank (bỏ qua khi NB = 1)
`i_dw_addr`	10	In	DMA	Địa chỉ word logic (bảng 4.1)
`i_dw_data`	32	In	DMA	
`o_dw_ready`	1	Out	DMA	Luôn bằng 1
Cổng đọc phía DMA (STORE_C)				
`i_dr_valid, i_dr_bank, i_dr_addr`	1, 1, 10	In	DMA	Đọc một word C, địa chỉ logic
`o_dr_ready`	1	Out	DMA	Luôn bằng 1
`o_dr_rvalid, o_dr_rdata`	1, 32	Out	DMA	Dữ liệu, 1 chu kỳ sau yêu cầu

Bảng 3.1: Port của `gemm_local_sram` (tiếp theo)

Port	Rộng	Chiều	Nối với	Mô tả
Bank phía tính (từ FSM)				
`i_comp_bank_a, _b, _bias, _c`	1×4	In	FSM	Bank array/VPU dùng; ổn định trong COMPUTE
`i_comp_active`	1	In	FSM	COMPUTE đang chạy (chỉ dùng cho assertion)
Cổng đọc A, B (từ array; spec array bảng 3-1)				
`i_a_rd_en, i_a_rd_row, i_a_rd_k`	1, 5, 5	In	Array	i_0 (bội của P), k
`o_a_rd_data`	8P	Out	Array	Lane r = $A[i_0 + r][k]$
`i_b_rd_en, i_b_rd_k, i_b_rd_col`	1, 5, 5	In	Array	k, j_0 (bội của P)
`o_b_rd_data`	8P	Out	Array	Lane c = $B[k][j_0 + c]$
Cổng BIAS và C (từ VPU; spec VPU bảng 3-1)				
`i_bias_rd_en, i_bias_rd_col`	1, 5	In	VPU	j_0
`o_bias_rd_data`	32P	Out	VPU	Lane c = $bias[j_0 + c]$
`i_c_wr_en, i_c_wr_fmt, i_c_wr_row, i_c_wr_col`	1, 1, 5, 5	In	VPU	Ghi một đoạn hàng
`i_c_wr_data, i_c_wr_mask`	32P, P	In	VPU	Bảng 3-3 spec VPU

Bảng 3.2: Giao tiếp

ID	Yêu cầu
SRM-I01	`o_dw_ready` = `o_dr_ready` = 1 hằng số. DMA vẫn phải tôn trọng bắt tay để giữ đúng SPM-05
SRM-I02	`o_a_rd_data`, `o_b_rd_data`, `o_bias_rd_data` hợp lệ ở đúng chu kỳ sau chu kỳ có `*_rd_en`; ở chu kỳ khác giá trị tùy ý. `o_dr_rvalid` là thanh ghi
SRM-I03	Lane ứng với hàng hoặc cột ngoài vùng hợp lệ của tác vụ mang nội dung tùy ý (SPM-03, SA-10)

4 Bố cục logic

Bảng 4.1: Địa chỉ word logic trong mỗi bank (SPM-02)

Buffer	Địa chỉ a	Nội dung word
A	{i[4:0], kw[2:0]}	Byte e = $A[i][4kw + e]$
B	{k[4:0], jw[2:0]}	Byte e = $B[k][4jw + e]$
BIAS	j[4:0]	$bias[j]$
C, INT32	{i[4:0], j[4:0]}	$C[i][j]$
C, INT8	{i[4:0], jw[2:0]}	Byte e = $Y[i][4jw + e]$ (dùng word 0 … 255)

5 Tổ chức vật lý và ánh xạ

Bảng 5.1: Sub-bank của mỗi bank

Buffer	Số sub-bank	Sâu × rộng	Sub-bank của a	Địa chỉ cục bộ
A	P	(256/P) × 32	i mod P	{i[4:LP], kw}
B	Q = P/4	(256/Q) × 32	jw mod Q	{k, jw » $\log_2$Q}
BIAS	P	(32/P) × 32	j mod P	j » LP
C	P	(1024/P) × 32, ghi theo byte	a mod P	a » LP

Bảng 5.2: Số RAM vật lý

Cấu hình	A	B	BIAS	C
P = 4, NB = 1	4 × 64	1 × 256	4 × 8	4 × 256
P = 8, NB = 1	8 × 32	2 × 128	8 × 4	8 × 128
P = 8, NB = 2	16 × 32	4 × 128	16 × 4	16 × 128

5.1 Truy cập phía tính

Bảng 5.3: Cổng phía tính (b = `i_comp_bank_*`)

ID	Yêu cầu
MAP-01	Đọc A (i_0, k): sub-bank r ($0 \le$ r < P) của A[b] đọc địa chỉ {i_0[4:LP], k[4:2]}. Lane r = byte k[1:0] của sub-bank r; k[1:0] được chốt để chọn byte ở chu kỳ sau
MAP-02	Đọc B (k, j_0): sub-bank s ($0 \le$ s < Q) của B[b] đọc {k, (j_0 » 2) » $\log_2$Q}. Lane c = byte (c mod 4) của sub-bank c/4
MAP-03	Đọc BIAS (j_0): sub-bank c của BIAS[b] đọc j_0 » LP; lane c = sub-bank c
MAP-04	Ghi C INT32 (i, j_0): lane c ghi sub-bank c của C[b] tại ({i, j_0} » LP), cả 4 byte, nếu `mask[c]`
MAP-05	Ghi C INT8 (i, j_0): word thứ w ($0 \le$ w < Q) của đoạn hàng có địa chỉ a = {i, j_0/4 + w}, nằm ở sub-bank s = (j_0/4 + w) mod P, địa chỉ a » LP. Sub-bank s nhận word w = (s − j_0/4) mod P nếu w < Q; byte e của nó = `i_c_wr_data[8(4w+e)+:8]`, byte enable e = `mask[4w+e]`

5.2 Truy cập phía DMA

Bảng 5.4: Cổng DMA

ID	Yêu cầu
MAP-06	Ghi `i_dw_*`: sub-bank và địa chỉ cục bộ theo bảng 5.1 với a = `i_dw_addr`, trong buffer `i_dw_buf`, bank `i_dw_bank`; cả 4 byte
MAP-07	Đọc `i_dr_*`: sub-bank (a mod P) của C[`i_dr_bank`], địa chỉ a » LP. Số sub-bank và bank được chốt để chọn dữ liệu ra ở chu kỳ sau

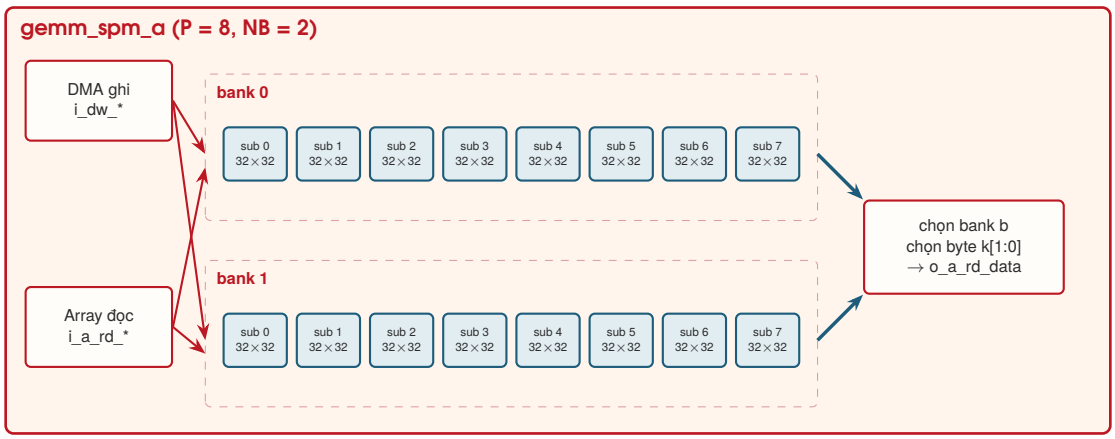
VÌ SAO KHÔNG BAO GIỜ XUNG ĐỘT

i_0 và j_0 là bội của P, nên P hàng của một đọc A rơi vào P sub-bank khác nhau, P cột của một đọc BIAS hoặc ghi C INT32 cũng vậy; Q word của một đọc B hoặc ghi C INT8 nằm liền nhau nên rơi vào Q sub-bank khác nhau. Ở INT8, {i, jw} = 8i + jw và 8i chia hết cho P, nên sub-bank chỉ phụ thuộc jw. Mô hình kiểm đủ mọi (i_0, k), (k, j_0), mọi đoạn hàng C, với P = 4, 8.

5.3 Chọn bên truy cập

Bảng 5.5: Mux cổng của mỗi sub-bank (X, b, s)

ID	Yêu cầu
MUX-01	`dma_sel` = lệnh DMA (ghi `i_dw_*` cho A, B, BIAS; đọc `i_dr_*` cho C) có buffer X, bank b và sub-bank s
MUX-02	`cmp_sel` = lệnh phía tính tương ứng (`i_a_rd_en`, `i_b_rd_en`, `i_bias_rd_en`, `i_c_wr_en` với byte enable khác 0 cho s) và `i_comp_bank_X` = b
MUX-03	RAM: `en=dma_selvcmp_sel`; địa chỉ, dữ liệu, byte enable lấy từ bên được chọn. Nếu cả hai (vi phạm, AS-02), DMA thắng
MUX-04	Không cần tín hiệu pha: quyền truy cập do bank và lịch của FSM bảo đảm (SB-01, SB-02 spec FSM)



Mỗi sub-bank: mux cổng (MUX-01 … 03). Ping-pong: DMA vào bank này, array đọc bank kia cùng chu kỳ.

Hình 5.1: Ví dụ buffer A với P = 8, NB = 2: 16 RAM 32 word

6 Module con

Bảng 6.1: Port của `gemm_spm_ram` (tham số DEPTH, BE)

Port	Rộng	Chiều	Mô tả
`clk`	1	In	
`i_en`, `i_we`	1, 1	In	Truy cập; ghi
`i_be`	4	In	Byte enable (BE = 0: bỏ qua, ghi cả word)
`i_addr`	$\lceil \log_2 \text{DEPTH} \rceil$	In	
`i_wdata`, `o_rdata`	32, 32	In, Out	`o_rdata` hợp lệ 1 chu kỳ sau `i_en` $\wedge \neg$`i_we`

Bảng 6.2: Module con theo buffer

Module	Nội dung
`gemm_spm_a`	NB × P RAM; MAP-01, MAP-06; chốt k[1:0] và bank cho mux byte
`gemm_spm_b`	NB × Q RAM; MAP-02, MAP-06
`gemm_spm_bias`	NB × P RAM (được hiện thực bằng thanh ghi vì chỉ 4 hoặc 8 word mỗi RAM); MAP-03, MAP-06
`gemm_spm_c`	NB × P RAM có byte enable; MAP-04, MAP-05, MAP-07; mux xoay word ở INT8; chốt sub-bank, bank cho `o_dr_rdata`

```
// gemm_spm_ram: RAM một cổng, đọc đồng bộ (tham khảo)
logic [31:0] mem [DEPTH];
always_ff @(posedge clk)
  if (i_en) begin
    if (i_we) begin
      for (int e = 0; e < 4; e++)
        if (!BE || i_be[e]) mem[i_addr][8*e +: 8] <= i_wdata[8*e +: 8];
    end else
      o_rdata <= mem[i_addr];
  end
```

7 Timing

Bảng 7.1: Thời điểm (t = chu kỳ có lệnh)

Cổng	Lệnh	Kết quả
A, B, BIAS đọc	t	dữ liệu ở t + 1
C ghi (VPU), A/B/BIAS ghi (DMA)	t	có trong RAM từ t + 1
C đọc (DMA)	t	`o_dr_rvalid` = 1, dữ liệu ở t + 1

8 Reset, hiện thực, kiểm chứng

Bảng 8.1: Reset

ID	Yêu cầu
RST-S01	`rst_n` = 0 hoặc `i_soft_clr`: `o_dr_rvalid` về 0. Thanh ghi chọn byte, bank, sub-bank và nội dung RAM không bị reset

Bảng 8.2: Hiện thực

ID	Yêu cầu
IMP-01	RAM viết theo mẫu `gemm_spm_ram` để công cụ suy ra block RAM hoặc LUTRAM có đọc đồng bộ. Không được dùng RAM đọc bất đồng bộ cho A, B, C (sai timing SRM-F02)
IMP-02	Không có mux tổ hợp từ `*_rd_en` tới dữ liệu ra; đường ra là: thanh ghi đầu ra RAM → mux bank → mux byte/lane
IMP-03	Mỗi buffer trong file riêng; NB và số sub-bank dựng bằng `generate`

Bảng 8.3: Assertion bắt buộc (SVA)

ID	Yêu cầu
AS-01	`i_dw_valid` → `i_dw_buf` ≠ 3 ∧ địa chỉ trong miền của buffer (A, B < 256; BIAS < 32)
AS-02	NB = 2: `i_dw_valid` ∧ `i_comp_active` → `i_dw_bank` ≠ `i_comp_bank_X` với X = `i_dw_buf`; và `i_dr_valid` ∧ `i_comp_active` → `i_dr_bank` ≠ `i_comp_bank_c`. NB = 1: `i_dw_valid` ∨ `i_dr_valid` → ¬`i_comp_active` (OWN-02)
AS-03	`i_a_rd_en` ∨ `i_b_rd_en` ∨ `i_bias_rd_en` ∨ `i_c_wr_en` → `i_comp_active`
AS-04	Không sub-bank nào có `dma_sel` ∧ `cmp_sel` cùng chu kỳ
AS-05	`i_a_rd_row`, `i_b_rd_col`, `i_bias_rd_col`, `i_c_wr_col` là bội của P khi có lệnh

KIỂM CHỨNG BẰNG MÔ HÌNH

`gemm_sram_model.py`: ánh xạ của bảng 5.1 là song ánh với P = 4, 8; 240 cấu hình ngẫu nhiên (M, N, K, bank, NB = 1, 2) nạp A, B, BIAS qua cổng DMA rồi đọc qua cổng array/VPU, ghi C INT32 và INT8 qua cổng VPU rồi đọc qua cổng DMA: mọi giá trị khớp và không truy cập song song nào chạm hai lần một sub-bank.